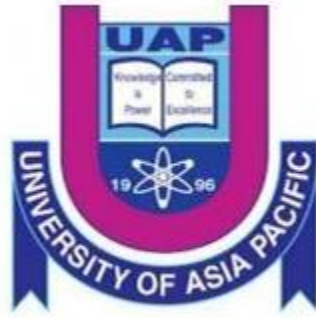




UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

EXPERIMENT: Lab 02 - SJF | Comparison with FCFS

Submitted by:

Name: Riya Mone

Student ID: 23101153

Section: C-1

Semester: 4-1

Submitted to:

Atia Rahman Orthi,

Lecturer,

University of Asia Pacific

Lab Report: SJF Comparison with FCFS

Problem Statement

Implement SJF (Non-Preemptive) and FCFS CPU scheduling algorithms for the given processes. Calculate Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average Turnaround Time and Average Waiting Time. Compare both algorithms to determine which performs better.

Objective

The objective of this lab is to understand the working procedure of Shortest Job First (SJF) Non-Preemptive scheduling and compare it with First Come First Serve (FCFS). The lab calculates CT, TAT and WT for each process, finds Average TAT and Average WT, and observes how the scheduling order affects system performance.

Given Data

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	2	5
P2	3	1
P3	0	2
P4	1	3
P5	1	6

Working Principle

SJF (Non-Preemptive): Among the processes that have already arrived, the process with the shortest Burst Time is selected first. Once a process starts execution, it continues until completion. After each completion, the available processes are checked again and the shortest one is selected.

FCFS: Processes are executed according to their arrival order. The process that arrives first gets the CPU first. If the CPU is idle, execution starts when the next process arrives.

For both algorithms: $TAT = CT - AT$ and $WT = TAT - BT$. The formulas are the same, but the values may be different because the execution order and completion time can be different.

Why SJF Can Perform Better

SJF gives priority to shorter available jobs. As a result, more processes can finish earlier and the average waiting time and average turnaround time are often reduced. In this experiment, SJF produces lower Average TAT and Average WT than FCFS.

Issues of SJF

Starvation: A process with a large Burst Time may wait for a long time if shorter processes continue to arrive.

Burst Time Prediction: SJF needs the CPU burst time or an estimate of it. In real systems, the exact future burst time is not always known.

Fairness: Short processes receive priority, so long processes may not get service as quickly as in FCFS.

FCFS and Starvation

FCFS normally avoids starvation because processes are served in first-come-first-serve order and every process eventually gets a chance. However, FCFS does not always reduce waiting time. A long process at the front can make many short processes wait; this is called the convoy effect.

Where FCFS and SJF Are Used

FCFS is useful in simple queues such as printer queues, basic batch processing, service/ticket queues and situations where arrival-order fairness is important.

SJF is useful in batch or job scheduling when the execution time can be estimated and reducing average waiting time is important.

Source Code

```
process = ["P1", "P2", "P3", "P4", "P5"]
at = [2, 3, 0, 1, 1]
bt = [5, 1, 2, 3, 6]
```

```
n = len(process)
```

```
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [0] * n
```

```
time = 0
completed = 0
```

```
while completed < n:
    x = -1
```

```
    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if x == -1 or bt[i] < bt[x]:
                x = i
```

```
    if x == -1:
        time = time + 1
    else:
        time = time + bt[x]
        ct[x] = time
        done[x] = 1
        completed = completed + 1
```

```
for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]
```

```

print("SJF RESULT")
print(f'{"Process":<10} {"AT":<6} {"BT":<6} {"CT":<6} {"TAT":<6} {"WT":<6}')
```

for i in range(n):

```

    print(f'{"process[i]":<10} {"at[i]":<6} {"bt[i]":<6} {"ct[i]":<6} {"tat[i]":<6} {"wt[i]":<6}')
```

sjf_avg_tat = sum(tat) / n
sjf_avg_wt = sum(wt) / n

print()
print("Average TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)

print()
print()

FCFS

```

processes = [
    ["P1", 2, 5],
    ["P2", 3, 1],
    ["P3", 0, 2],
    ["P4", 1, 3],
    ["P5", 1, 6]
]
```

processes.sort(key=lambda x: x[1])

time = 0
total_tat = 0
total_wt = 0

print("FCFS RESULT")
print(f'{"Process":<10} {"AT":<6} {"BT":<6} {"CT":<6} {"TAT":<6} {"WT":<6}')

for p in processes:

```

    pid = p[0]
    arrival = p[1]
    burst = p[2]

    if time < arrival:
        time = arrival

    time = time + burst
    completion = time
    turnaround = completion - arrival
    waiting = turnaround - burst

    total_tat = total_tat + turnaround
    total_wt = total_wt + waiting

    print(f'{"pid":<10} {"arrival":<6} {"burst":<6} {"completion":<6} {"turnaround":<6} {"waiting":<6}')
```

fcfs_avg_tat = total_tat / len(processes)
fcfs_avg_wt = total_wt / len(processes)

print()
print("Average TAT =", fcfs_avg_tat)
print("Average WT =", fcfs_avg_wt)

print()
print()

```

if sjf_avg_tat < fcfs_avg_tat:
    print("SJF is better TAT than FCFS")
else:
    print("FCFS is better TAT than SJF")

```

```

if sjf_avg_wt < fcfs_avg_wt:
    print("SJF is better WT than FCFS")
else:
    print("FCFS is better WT than SJF")

```

Output

SJF RESULT

Process	AT	BT	CT	TAT	WT
P1	2	5	11	9	4
P2	3	1	6	3	2
P3	0	2	2	2	0
P4	1	3	5	4	1
P5	1	6	17	16	10

Average TAT = 6.8

Average WT = 3.4

FCFS RESULT

Process	AT	BT	CT	TAT	WT
P3	0	2	2	2	0
P4	1	3	5	4	1
P5	1	6	11	10	4
P1	2	5	16	14	9
P2	3	1	17	14	13

Average TAT = 8.8

Average WT = 5.4

SJF is better TAT than FCFS

SJF is better WT than FCFS

Result Summary

SJF Result

Process	AT	BT	CT	TAT	WT
P1	2	5	11	9	4
P2	3	1	6	3	2
P3	0	2	2	2	0

P4	1	3	5	4	1
P5	1	6	17	16	10

Average TAT = 6.8

Average WT = 3.4

FCFS Result

Process	AT	BT	CT	TAT	WT
P3	0	2	2	2	0
P4	1	3	5	4	1
P5	1	6	11	10	4
P1	2	5	16	14	9
P2	3	1	17	14	13

Average TAT = 8.8

Average WT = 5.4

Comparison between SJF and FCFS

Point	SJF	FCFS
Selection Method	Shortest BT among arrived processes	First arrival first
Average TAT in this experiment	6.8	8.8
Average WT in this experiment	3.4	5.4
Starvation	Possible for long jobs	Normally avoided
Convoy Effect	Less likely than FCFS	Possible
Burst Time Knowledge	Required/estimated	Not required
Fairness	Short jobs get priority	Arrival-order fairness
Complexity	More decision making	Simple
Better in this experiment	Yes	No

Discussion

In this experiment, SJF Non-Preemptive and FCFS CPU scheduling algorithms were implemented using the same set of processes. SJF selected the shortest Burst Time from the processes that were already available, while FCFS followed the order of arrival. Because the scheduling order was different, the Completion Time, Turnaround Time and Waiting Time were also different even though the TAT and WT formulas were the same. The SJF algorithm produced an Average TAT of 6.8 and an Average WT of 3.4. FCFS produced an Average TAT of 8.8 and an Average WT of 5.4. Therefore, for the given data, SJF performed better because it reduced both average turnaround time and average waiting time.

However, SJF has some limitations. Long jobs may suffer from starvation if short jobs keep arriving, and the algorithm requires the Burst Time to be known or estimated. FCFS is simpler and normally avoids starvation, but it may suffer from the convoy effect when a long process makes shorter processes wait.

Conclusion

The experiment successfully implemented and compared SJF Non-Preemptive and FCFS scheduling algorithms. The required CT, TAT, WT, Average TAT and Average WT were calculated correctly. For the given processes, SJF performed better than FCFS because SJF had lower Average TAT and Average WT. The experiment also showed that SJF is efficient for reducing average waiting time, while FCFS is simpler and provides arrival-order fairness.